



香山处理器 Tutorial

李衍君¹ 徐之皓¹ 陈泱宇² 杨泽辰¹

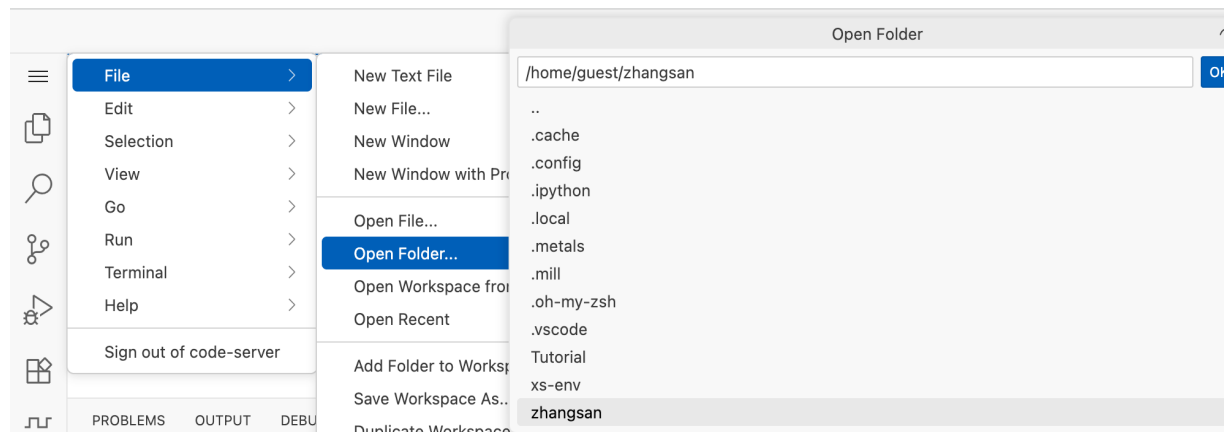
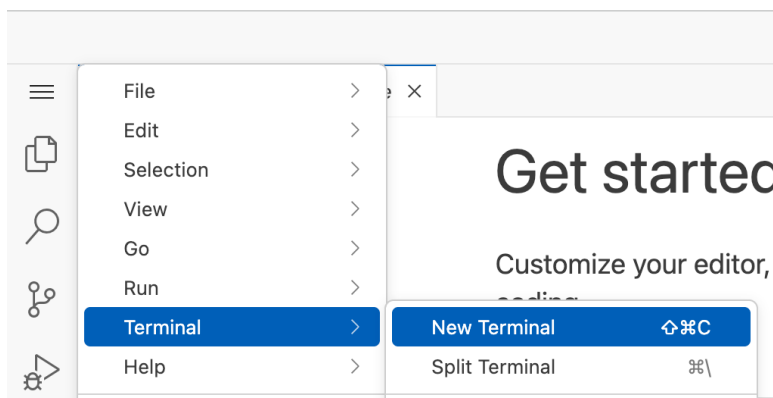
¹中国科学院计算技术研究所 ²重庆大学

2025 年 7 月 16 日



服务器登录

1. 使用浏览器访问 <https://xs.oscc.cc>
2. 打开 Terminal
3. 将 `/opt/xs-env` 复制到以你的名字命名的文件夹
 - `cp -r /opt/xs-env ~/zhangsan`
4. 打开自己名字命名的文件夹，这是属于你自己的非共享工作区
5. 跟随 <https://tutorial.xiangshan.cc> 操作



从零开始学习香山微架构

- 香山现在有了与开发同步的设计文档
 - Github: <https://github.com/OpenXiangShan/XiangShan-Design-Doc>
 - 网站: <https://docs.xiangshan.cc/projects/design/>
- Markdown 格式, 自动部署

 香山开源处理器设计文档

搜索

XiangShan-Design-Doc
☆ 29 7

香山开源处理器设计文档

前言

本文档为《香山开源处理器设计文档》，详细介绍了 昆明湖 V2R2 的微架构设计实现细节。

您可从以下渠道获取最新版的本文档：

- 网页版: <https://docs.xiangshan.cc/projects/design/>
- PDF 文件: <https://github.com/OpenXiangShan/XiangShan-Design-Doc/releases>

您可访问香山文档网站 docs.xiangshan.cc 获取更多文档。

目录
声明
版本信息

XiangShan-Design-Doc Public

Edit Pins Watch 5 Fork 7 Star 29

master 10 Branches 0 Tags Go to file Code

lewiszh docs(HPM): fix the error of event number hp... 41c955f · 2 months ago 144 Commits

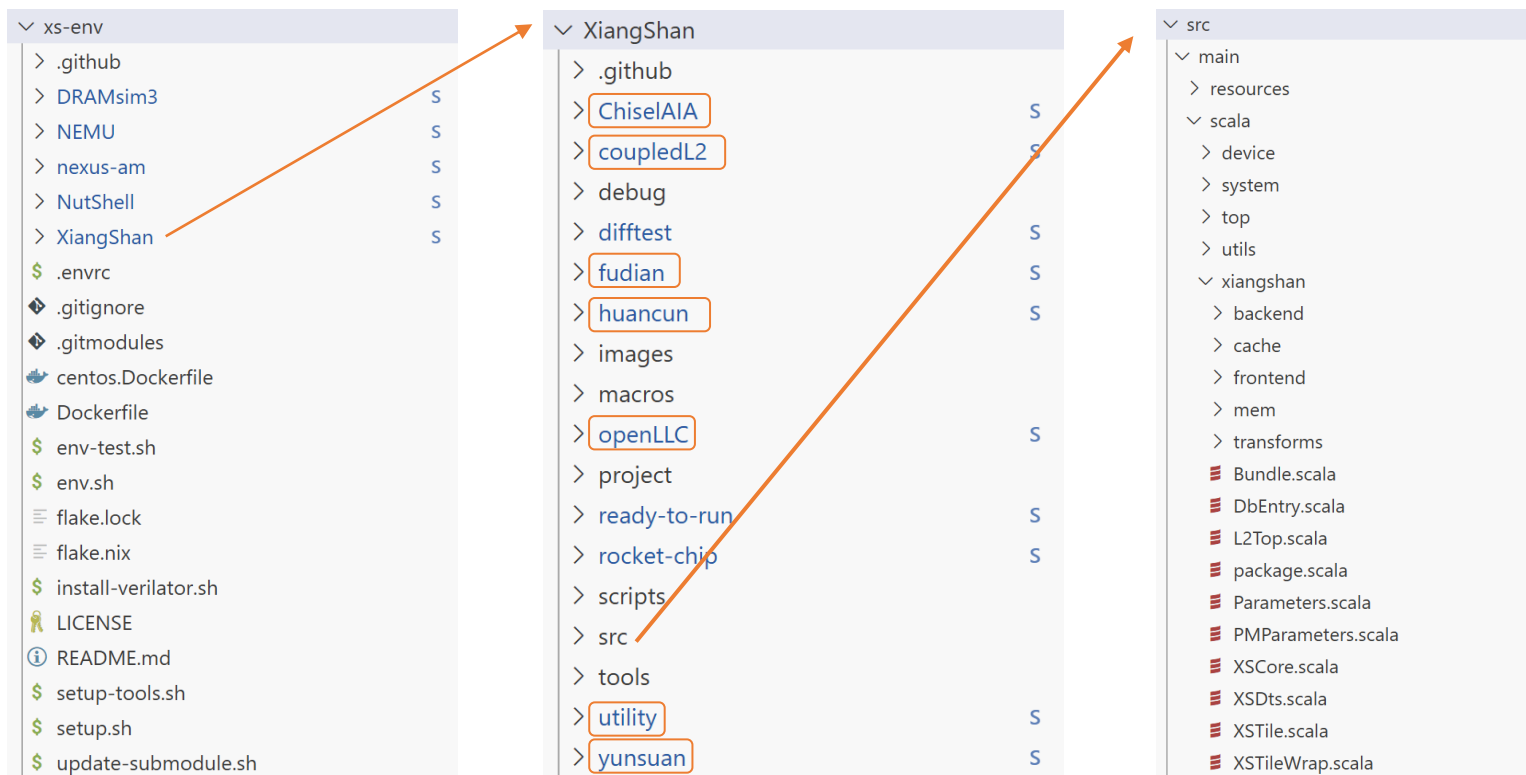
github	chore: create CODEOWNERS (#50)	4 months ago
docs	docs(HPM): fix the error of event number...	2 months ago
tools/lint	chore: add markdown linter	6 months ago
utils @ 45de1f3	build: bump utils	4 months ago
.gitattributes	chore: add gitattributes	6 months ago
.gitignore	build: bump utils to support several featu...	5 months ago

About
Documentation for XiangShan Design
docs.xiangshan.cc/projects/design/
Readme
CC-BY-4.0 license
Activity
Custom properties
29 stars
5 watching
7 forks
Report repository



一切开始的地方

- 香山仓库: <https://github.com/OpenXiangShan/XiangShan>
- 开发环境套件: <https://github.com/OpenXiangShan/xs-env>
- 服务器: xs-env 文件夹





配置香山

- 香山具有数百个可自由定制的参数
 - 处理器核参数，如 ROB 大小、Cache 相联度等，通过 Configs.scala 配置
 - SoC 参数，如 PMA 配置、L2 与 L3 Cache 大小等，通过 yaml 文件传递
- DefaultConfig：香山默认配置
- MinimalConfig：经过裁剪过的香山
 - 仍然为乱序超标量处理器
 - 包含 L1 和 L3 Cache，不包含 L2 Cache
 - 编译较快，占用资源较小，适合日常开发



获取 Verilog 与 emu

- Scala 执行生成 Verilog, 可选:
 - CONFIG, 香山配置, 如 MinimalConfig
- Verilator 读取 Verilog, 生成可执行的仿真程序 emu, 可选:
 - EMU_THREADS, 仿真线程数
 - EMU_TRACE, 是否生成波形
 - WITH_DRAMSIM, 是否启用模拟 DRAM 仿真的 DRAMSim3
 - WITH_CHISELDB, 是否开启 ChiselDB
 - WITH_CONSTANTIN, 是否开启 ConstantIn
- MinimalConfig 参考配置: 16 核心, 80G 物理内存
- 需要约 20 分钟

要在香山上执行什么程序?

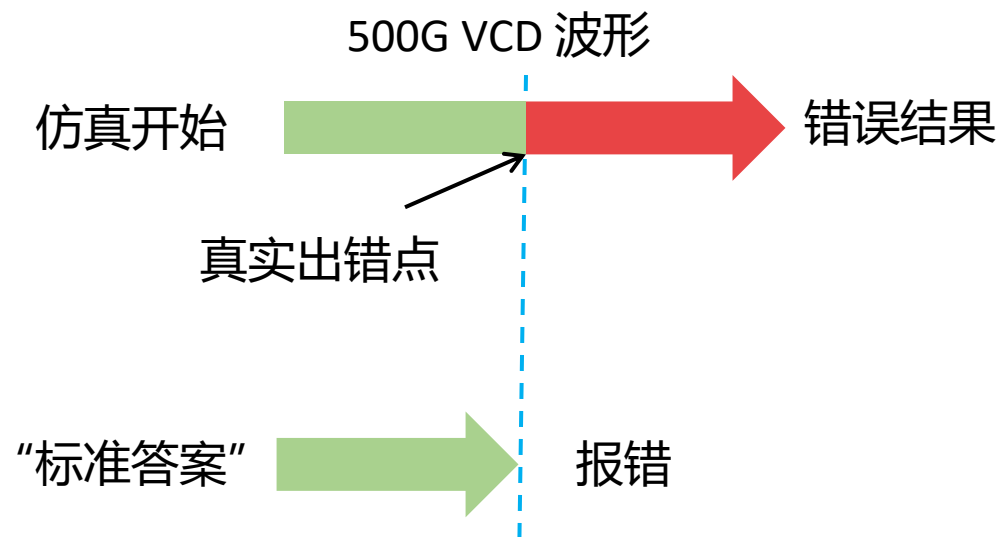
- 香山是一个裸金属 (bare metal) 机器
- 操作系统是运行在裸机上的, 但应用程序不是
- Linux 等操作系统过于重量级、不便于快速测试迭代
- 操作系统、编译器等需要为应用程序提供:
 - 库函数
 - 运行时环境
 - 异常处理

Nexus-AM: 裸机运行时环境

- 在没有操作系统的情况下敏捷地生成工作负载
- 为裸机（如香山）提供运行时框架
- 轻量且易用
- 实现了基本的系统调用接口和异常处理
- 支持多种 ISA 和配置

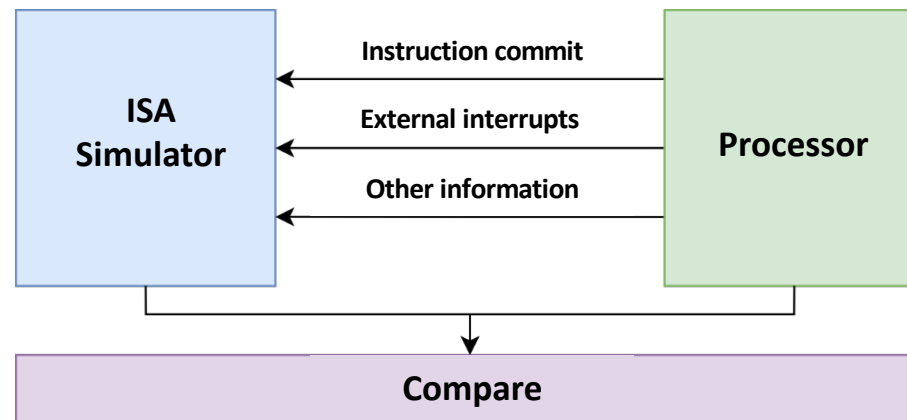
错误定位

- 程序结果错误，如何定位错误？
 - 真正的错误远在很久之前
 - 波形文件非常大，无法定位出错位置
 - 哪些地方是对的，哪些地方是错的？
- 希望：处理器一旦出错就立刻能知道
 - “抄作业”
 - 什么是“标准答案”？



DiffTest: 差分测试协同仿真框架

- 处理器指令提交
- 模拟器执行相同指令
- 对比体系结构状态，决定继续或中止
- 作为处理器验证的基础设施
 - 向 HDL (如 Chisel/Verilog) 提供 API 接口
 - 支持 RTL 模拟器 (如 Verilator, VCS)
 - 支持 RISC-V ISS (如 NEMU, Spike) 作为参考设计
 - 支持应用于硬件仿真加速器
- SMP-DiffTest: SMP 处理器上的协同仿真
 - SMP Linux kernel 和多线程程序
 - 在线检查 cache 一致性和内存一致性



NEMU: 高效的 ISA 参考设计

- 与 Spike 类似的指令集模拟器
- 经过优化，性能与 QEMU 接近
- 提供一系列 API，支持 difftest
- 支持 RVA23，默认配置和 XiangShan 对齐

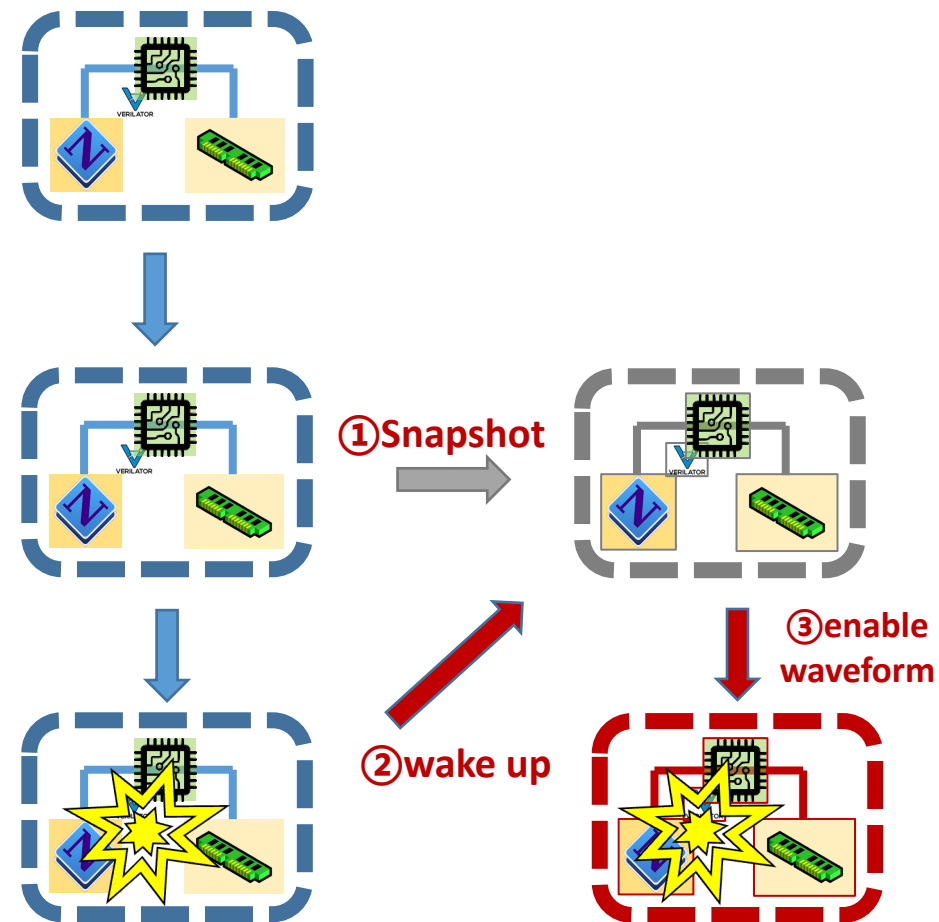


仿真时间长、波形大

- 关闭波形：速度较快，无法排除错误
- 开启波形：速度非常慢，波形文件巨大
- 我们需要某种检查点机制
 - 检查点：保存当前所有状态，以便下次从检查点继续
- fork 可以复制一个进程
- 仿真程序也是进程

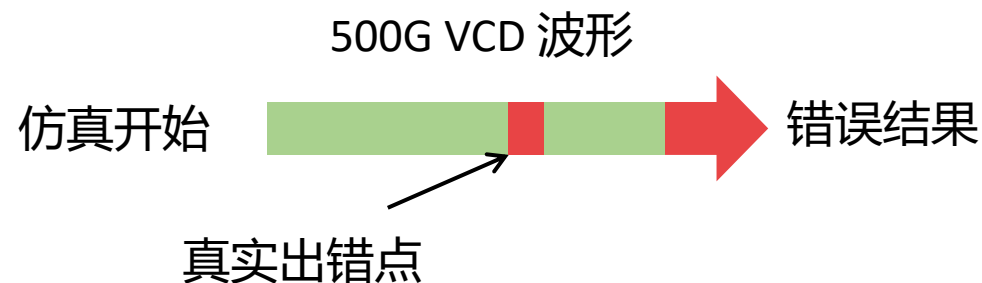
🏔️ LightSSS: 一种基于内存的轻量级仿真快照

- 使用 fork 对仿真进程做快照
- 具有良好的移植性和扩展性
 - 可以对任何外部模块做快照 (如 C++)
 - 无需理解外部模块的细节
- 快照开销低
 - Linux kernel 存在 CoW 机制
 - 生成快照只需要约 500us
 - 远低于 Verilator 提供的 RTL 快照开销



❄️ 错误定位 2.0 与性能分析

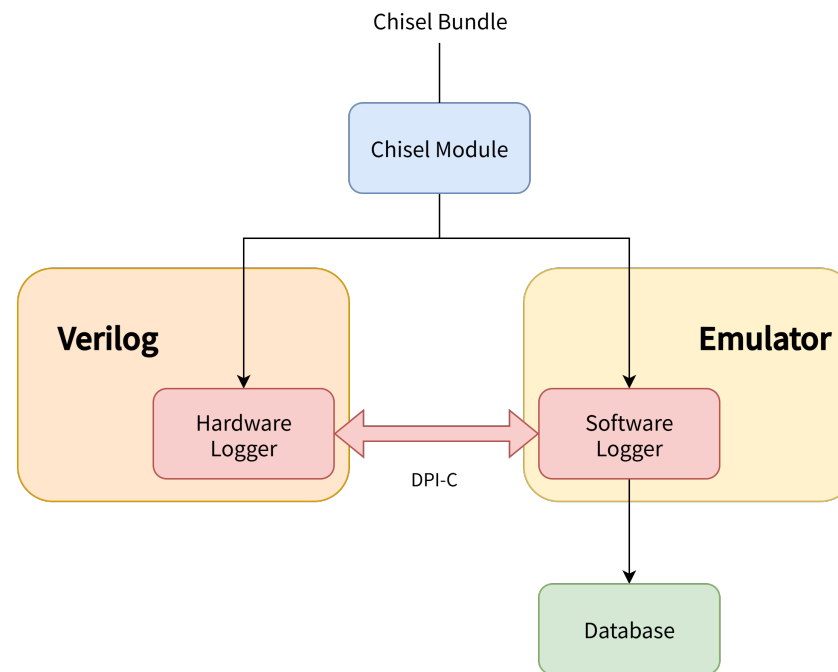
- 功能调试/性能分析需要trace
 - 仿真波形
 - Chisel print → verilog \$display



- 类比 C 和汇编，Chisel → Verilog 丢失部分高层语义信息
- 纯波形、日志难以分析，占用磁盘空间巨大
- 如何结构化存储 trace？如何保持高层语义？

ChiselDB: 结构化数据库

- 在硬件层面的模块接口间插入探针
- 硬件侧每周期记录硬件事件
- 软件侧生成对应事件解析器
- 使用 DPI-C 在软硬件之间传输数据
- 软件侧将数据可持久化存储到数据库
 - 支持 SQL 查询
 - 对数据进行深入分析



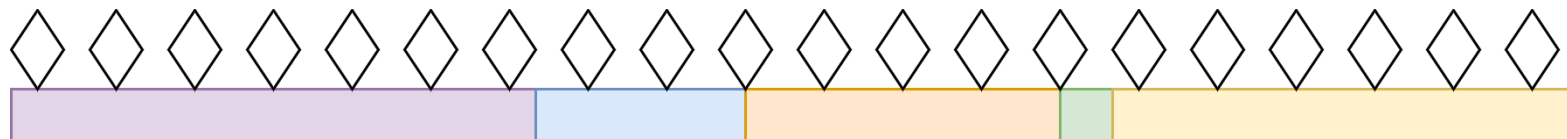
性能评估

- 仿真环境下完整运行 SPEC2006 需要 958 年
- FPGA 完整 SPEC2006 需要 7 天
- 能否把串行的程序切分成并行执行？
- 回忆 LightSSS：只要有完整状态，就可以从检查点继续运行

	RTL-simulation	FPGA
Compile	20 minutes	5 hours
Simulation	958 years@2KHz (2K CPS)	7 days@100MHz

Checkpoint 与性能分析

- Uniform Checkpoint: 均一采样



- Simpoint Checkpoint: 聚类分析, 带权重采样



	RTL-simulation	FPGA	RTL-Simulation w/ Checkpoint
Compile	20 minutes	5 hours	20 minutes + one-time checkpoint
Simulation	958 years@2KHz (2K CPS)	7 days@100MHz	5.5 hours with enough x86 servers



性能瓶颈在哪儿?

- 需要完备的性能计数器
- 需要有方法来指导数据分析

XSPerf: 细粒度的性能瓶颈分析

- 香山提供了全面的性能计数器
- 计数器分为三种
 - Accumulation: 基础的累加型计数器 (log 打印)
 - Histogram: 统计数值分布 (log 打印)
 - Rolling: 采集片段计数以分析性能变化 (ChiselDB 存储)

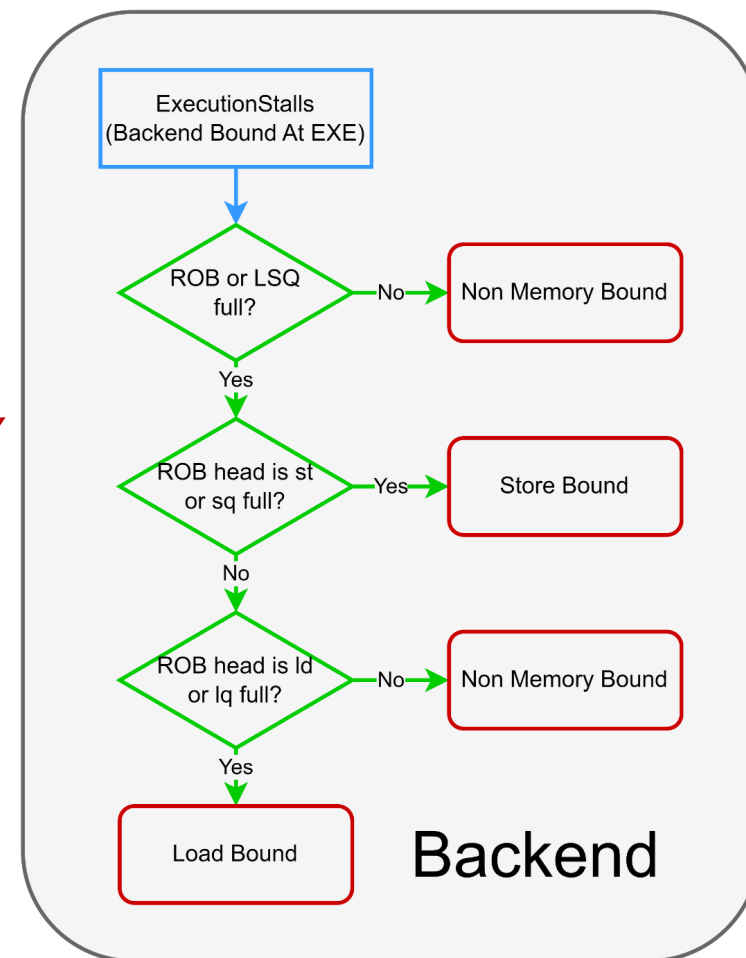
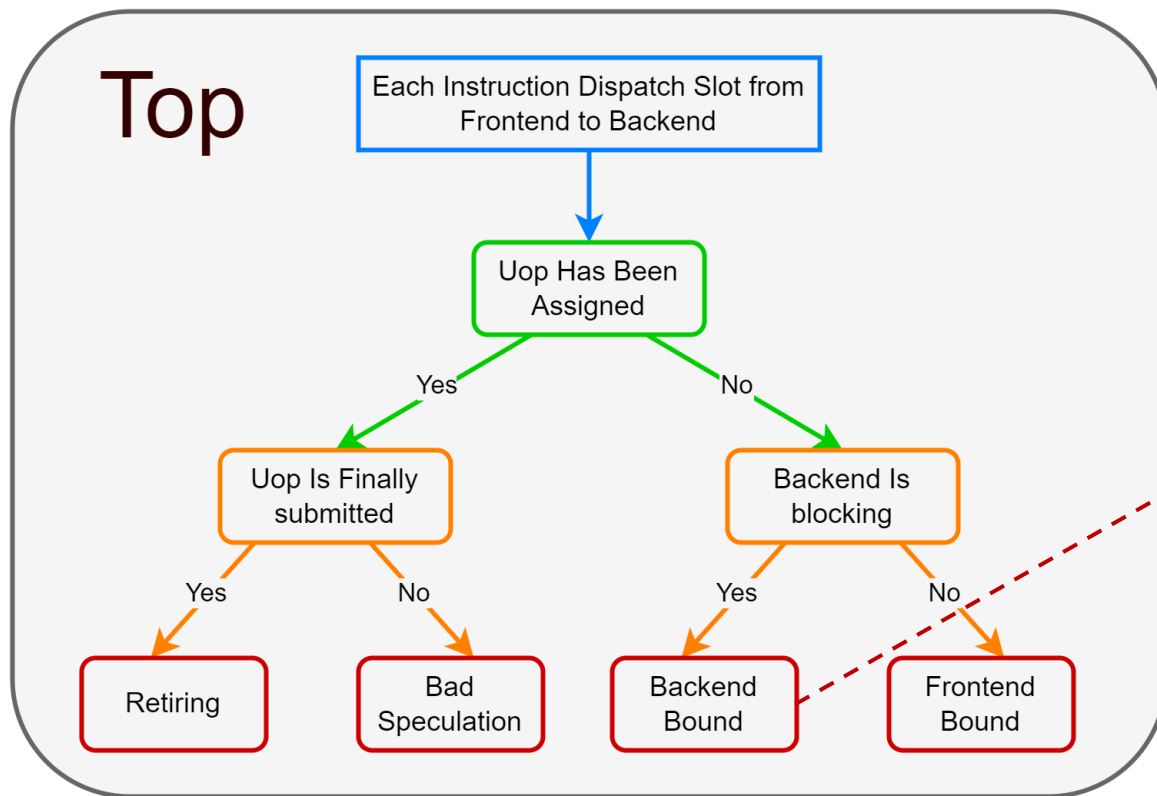


Topdown 性能分析方法

- 将分散的性能事件有层次地组织起来
- 精确计算性能事件对处理器性能的影响
- RTL 与 GEM5 均实现了完善的 Topdown 方法
 - 针对 RISC-V 指令集进行优化适配
 - 针对香山的微架构进行性能计数器优化
 - 进一步完善 Topdown 模块的层次化设计
 - 不会造成性能事件的重复或遗漏
 - 不需要提前假设性能事件的阻塞周期

🏔️ Topdown 性能分析方法

- 以后端为例

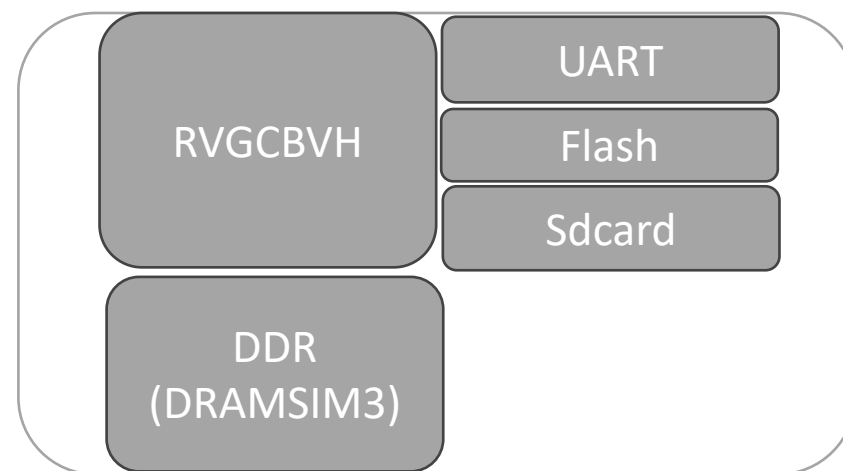




XS-Gem5

- 基于开源的 GEM5 模拟器构建分支
- 仿真速度: ~100 K Cycles/s (单线程)
 - Verilator 仿真香山 RTL : ~5 K Cycles/s (一个CCX, 8线程)
- 让快速的设计空间探索成为可能

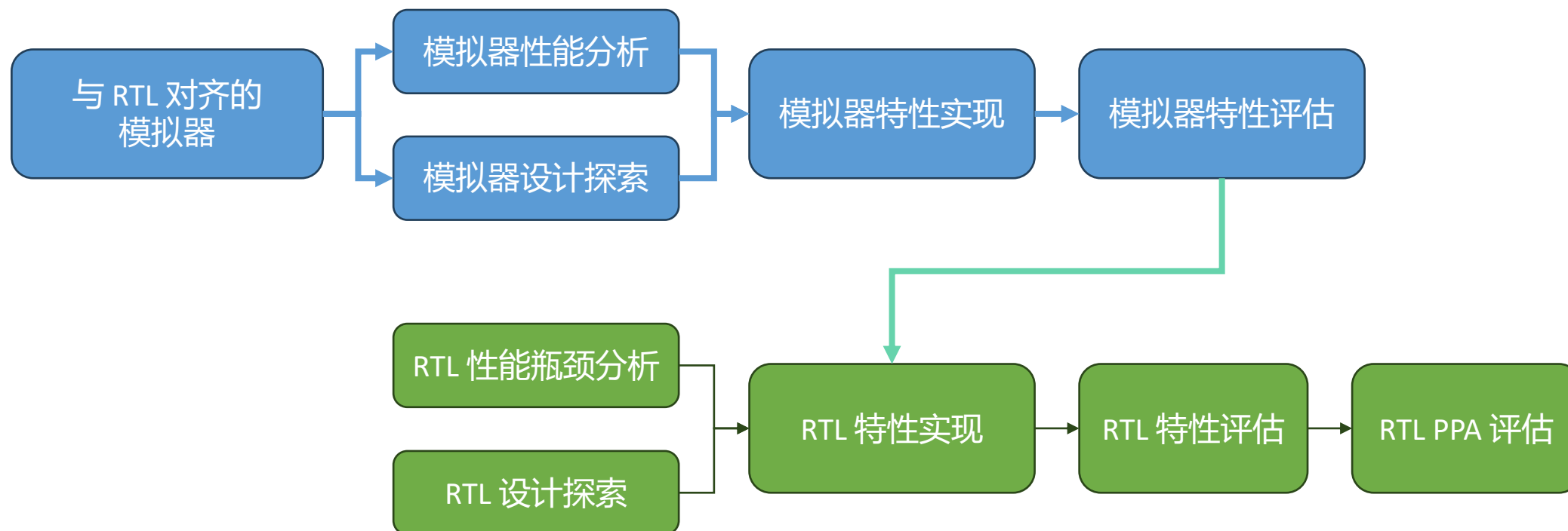
- 全系统模拟，包括：
 - RVGCpt: XiangShan's RISC-V generic checkpoint
 - 在线差分测试 (with NEMU / Spike)
- 支持 SimPoint 快速评估 SPEC CPU 2006 总分
- 前端：解耦式前端，与微架构对齐
- 后端 / 缓存：延迟 / 吞吐对齐
- 内存子系统
 - 预取器
 - 并行 PTW / L2 TLB / TLB 预取
 - 内存带宽和内存控制器对齐
 - dramsim3 支持





架构模拟器上的性能探索工作流程

- 基于架构模拟器的探索结果应能为香山的性能迭代提供指导
- 经过对齐的模拟器得出的结果更有可能在 RTL 上得到验证
- 模拟器指导的性能迭代工作流程：



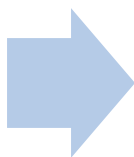


从零开始学习香山GEM5模拟器

- 主仓库地址: <https://github.com/OpenXiangShan/GEM5>
- 文档地址: <https://xs-gem5.readthedocs.io/>
- 文档地址可以在仓库地址找到

让我们动动手尝试一些基本的事情

在 GEM5 上
运行裸机应用



在 GEM5 上
查看性能计数器

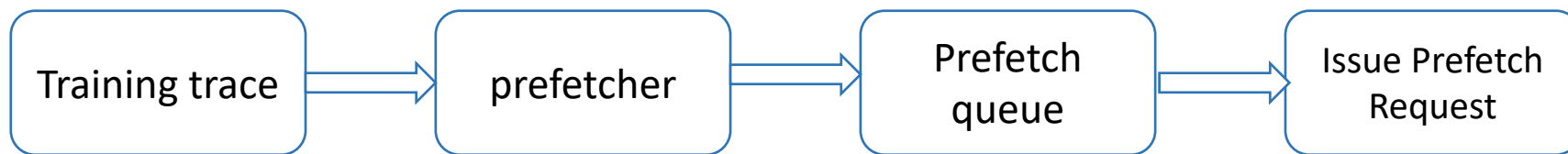


尝试 SPECCPU
算分脚本



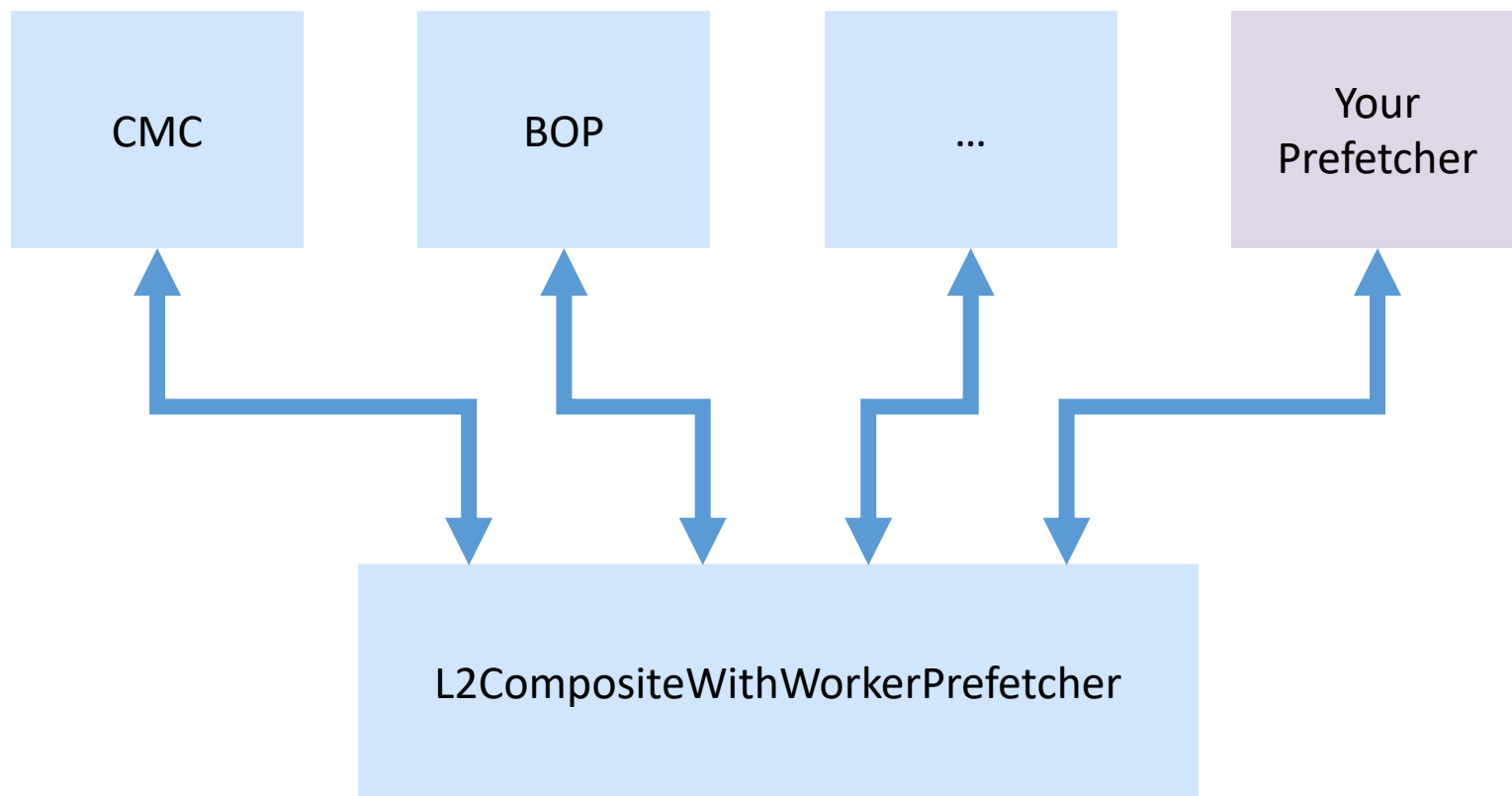
架构探索案例：如何添加一个预取器

- 举个栗子：集成 Despacito Stream 预取器 (PR #399)
- 在XS-GEM5中，预取执行被组织成几个组件。添加新的预取器涉及以下步骤：
 - 定义：在 prefetcher.py 中定义新的预取器
 - 配置预取器开关：在配置系统中设置预取器是否默认启用
 - 声明：在模拟器的预取框架中声明预取器
 - 实现：实现预取算法的逻辑
 - 调用：通过在现有预取框架中注册调用逻辑，将预取器集成到执行流程中





架构探索案例：如何添加一个预取器





架构探索案例：如何添加一个预取器

- 1. 在 prefetcher.py 中定义新的预取器
 - 预取器规格：
 - 指定预取距离、触发条件等关键参数。
 - 替换策略：
 - 在预取器中实现内部管理策略，例如条目替换或优先级处理。

```
diff --git a/src/mem/cache/prefetch/Prefetcher.py b/src/mem/cache/prefetch/Prefetcher.py
index d2cce7eb4d..a566c2a648 100644
--- a/src/mem/cache/prefetch/Prefetcher.py
+++ b/src/mem/cache/prefetch/Prefetcher.py
@@ -929,6 +929,57 @@ class CMCPrefetcher(QueuedPrefetcher):
     "Enable prefetch database"
 )

+class DespacitoStreamPrefetcher(QueuedPrefetcher):
+    type = "DespacitoStreamPrefetcher"
+    cxx_class = "gem5::prefetch::DespacitoStreamPrefetcher"
+    cxx_header = "mem/cache/prefetch/despacito_stream.hh"
+
+    use_virtual_addresses = False
+
+    class XSCompositePrefetcher(QueuedPrefetcher):
+        type = "XSCompositePrefetcher"
+        cxx_class = 'gem5::prefetch::XSCompositePrefetcher'
+
+@@ -1108,9 +1159,12 @@ class L2CompositeWithWorkerPrefetcher(CompositeWithWorkerPrefetcher):
+     bop_small = Param.BOPPrefetcher(SmallBOPPrefetcher(is_sub_prefetcher=True),
+                                     "Small BOP used in composite prefetcher")
+     despacito_stream = Param.DespacitoStreamPrefetcher(DespacitoStreamPrefetcher(is_sub_prefetcher=True),
+                                                         "DespacitoStream used in composite prefetcher")
+
+     enable_bop = Param.Bool(False, "Enable BOP")
+     enable_cdp = Param.Bool(True, "Enable CDP")
+     enable_sno = Param.Bool(False, "Enable SNO")
+     enable_despacito_stream = Param.Bool(True, "Enable despacito stream")
+
+    class L3CompositeWithWorkerPrefetcher(CompositeWithWorkerPrefetcher):
+        type = 'L3CompositeWithWorkerPrefetcher'
```

架构探索案例：如何添加一个预取器

- 2. 在 prefetcherConfig.py 中启用或禁用预取器

```
diff --git a/configs/common/PrefetcherConfig.py b/configs/common/PrefetcherConfig.py
index 9467763251..20d873d3da 100644
--- a/configs/common/PrefetcherConfig.py
+++ b/configs/common/PrefetcherConfig.py
@@ -65,6 +65,7 @@ def create_prefetcher(cpu, cache_level, options):
     prefetcher.enable_cmc = False
     prefetcher.enable_bop = True
     prefetcher.enable_cdp = False
+    prefetcher.enable_despacito_stream = False
     prefetcher.bop_large = XSVirtualLargeBOP(is_sub_prefetcher=True)
     prefetcher.bop_small = XSPhysicalSmallBOP(is_sub_prefetcher=True)
     if options.l1_to_l2_pf_hint:
```

架构探索案例：如何添加一个预取器

- 3. 在 SConscript 中声明预取器

```
diff --git a/src/mem/cache/prefetch/SConscript b/src/mem/cache/prefetch/SConscript
index c77c5494ad..61756a1bf6 100644
--- a/src/mem/cache/prefetch/SConscript
+++ b/src/mem/cache/prefetch/SConscript
@@ -37,7 +37,7 @@ SimObject('Prefetcher.py', sim_objects=[
    'SignaturePathPrefetcherV2', 'AccessMapPatternMatching', 'AMPMPrefetcher',
    'DeltaCorrelatingPredictionTables', 'DCPTPrefetcher',
    'IrregularStreamBufferPrefetcher', 'SlimAMPMPrefetcher',
-   'WorkerPrefetcher',
+   'WorkerPrefetcher', 'DespacitoStreamPrefetcher',
    'BOPPrefetcher', 'SB00EPrefetcher', 'STeMSPrefetcher', 'PIFPrefetcher', 'IPCPrefetcher',
    'CompositeWithWorkerPrefetcher', 'L2CompositeWithWorkerPrefetcher'])

@@ -58,6 +58,7 @@ DebugFlag('CDPHotVpns')
    DebugFlag('CDPdebug')
    DebugFlag('CDPdepth')
    DebugFlag('CMCPrefetcher')
+   DebugFlag('DespacitoStreamPrefetcher')

    Source('access_map_pattern_matching.cc')
    Source('base.cc')
@@ -86,4 +87,5 @@ Source('worker.cc')
    Source('cmc.cc')
    Source('composite_with_worker.cc')
    Source('l2_composite_with_worker.cc')
+   Source('despacito_stream.cc')
```



架构探索案例：如何添加一个预取器

- 4. 实现预取算法
 - 在源文件中编写预取器的核心逻辑：
 - src/mem/cache/prefetch/prefetch1.cc
 - src/mem/cache/prefetch/prefetch1.hh

```
> src/mem/cache/prefetch/despacito_stream.cc
```



```
> src/mem/cache/prefetch/despacito_stream.hh
```





架构探索案例：如何添加一个预取器

• 5. 调用 L1/L2 预取框架中的预取器

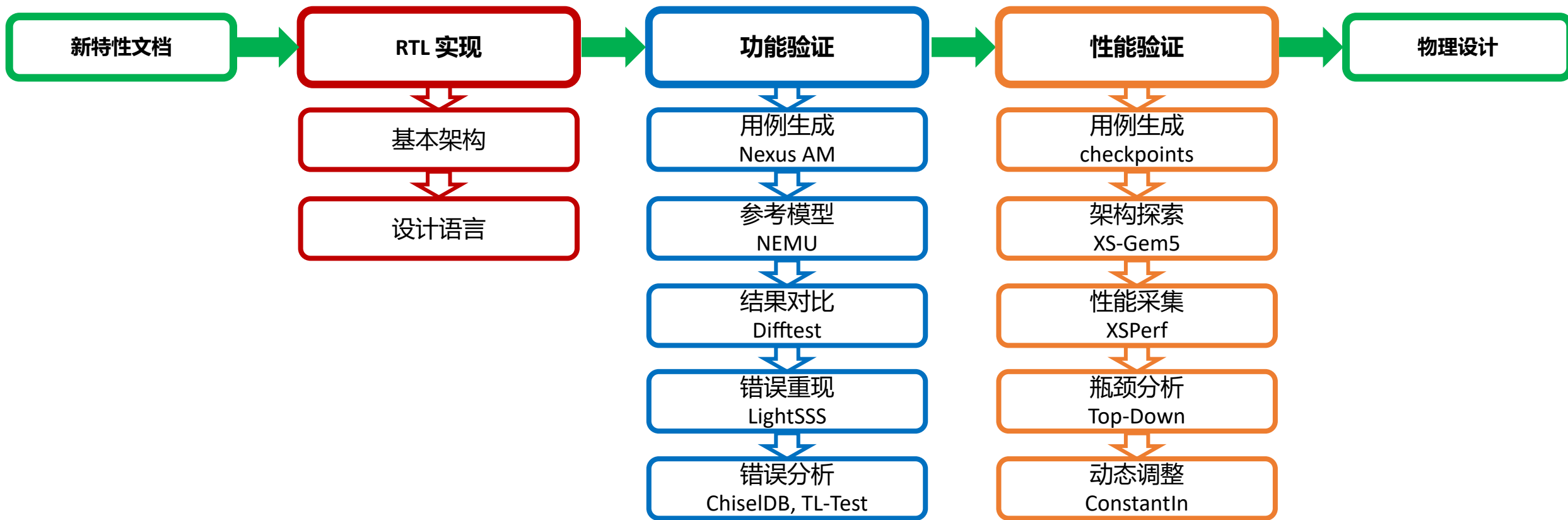
```
diff --git a/src/mem/cache/prefetch/l2_composite_with_worker.cc b/src/mem/cache/prefetch/l2_composite_with_worker.cc
index 31ba47bf01..b7ede79a17 100644
--- a/src/mem/cache/prefetch/l2_composite_with_worker.cc
+++ b/src/mem/cache/prefetch/l2_composite_with_worker.cc
@@ -16,14 +16,17 @@ L2CompositeWithWorkerPrefetcher::L2CompositeWithWorkerPrefetcher(const L2Composi
    largeBOP(p.bop_large),
    smallBOP(p.bop_small),
    cmc(p.cmc).
+   despacitoStream(p.despacito_stream),
+   enableBOP(p.enable_bop),
+   enableCDP(p.enable_cdp),
-   enableCMC(p.enable_cmc)
+   enableCMC(p.enable_cmc),
+   enableDespacitoStream(p.enable_despacito_stream)
{
    cdp->pfLRUFilter = &pfLRUFilter;
    largeBOP->filter = &pfLRUFilter;
    smallBOP->filter = &pfLRUFilter;
    cmc->filter = &pfLRUFilter;
+   despacitoStream->filter = &pfLRUFilter;
    cdp->parentId = p.sys->getRequestorId(this);
}

@@ -65,6 +68,9 @@ L2CompositeWithWorkerPrefetcher::calculatePrefetch(const PrefetchInfo &pfi, std:
    largeBOP->calculatePrefetch(pfi, addresses, late && pf_source == PrefetchSourceType::HWP_BOP);
    smallBOP->calculatePrefetch(pfi, addresses, late && pf_source == PrefetchSourceType::HWP_BOP);
}
+   if (enableDespacitoStream) {
+       despacitoStream->calculatePrefetch(pfi, addresses, late && pf_source == PrefetchSourceType::DespacitoStream);
+   }
}

void
@@ -121,6 +127,7 @@ L2CompositeWithWorkerPrefetcher::setParentInfo(System *sys, ProbeManager *pm, Ca
    largeBOP->setParentInfo(sys, pm, _cache, blk_size);
    smallBOP->setParentInfo(sys, pm, _cache, blk_size);
    cmc->setParentInfo(sys, pm, _cache, blk_size);
+   despacitoStream->setParentInfo(sys, pm, _cache, blk_size);
    L2CompositeWithWorkerPrefetcher::setParentInfo(sys, pm, _cache, blk_size);
}
```



总结: 香山敏捷开发流程与工具



感谢对香山的关注！



香山开源技术讨论群

群号 879550595